Terraform для PostgreSQL DBA: От ручного управления к Infrastructure as Code

Дополнительный материал для курса PostgreSQL Advanced

Дмитрий Золотов

2026-02-13

1 Terraform для PostgreSQL DBA: От ручного управления к Infrastructure as Code
1.1 Оглавление
1.2 Зачем DBA изучать Terraform
1.2.1 Мотивация: что изменится в вашей работе
1.2.2 Что даёт Terraform
1.2.3 Реальные кейсы из индустрии
1.2.4 Типичные боли DBA, которые решает Terraform
1.3 Infrastructure as Code: парадигма управления инфраструктурой
1.3.1 Что такое IaC и почему это важно
1.3.2 Сравнение подходов: Императивный vs Декларативный
1.3.3 Аналогия для DBA: Репликация
1.3.4 Типы IaC-инструментов
1.4 Terraform: основы для начинающих
1.4.1 Установка Terraform
1.4.2 Структура проекта Terraform
1.4.3 Ваш первый Terraform-проект: Создание базы данных и роли
1.4.4 Workflow: Команды Terraform
1.4.5 Типичные ошибки новичков
1.5 Управление PostgreSQL в облаках
1.5.1 AWS RDS: Создание и управление PostgreSQL

1.5.2 Azure Database for PostgreSQL

1.6 Terraform PostgreSQL Provider: управление внутри базы

1.6.1 Что умеет PostgreSQL Provider

1.6.2 Управление схемами

1.6.3 Управление расширениями

1.6.4 Сложный пример: Row-Level Security (RLS)

1.7 Модули и переиспользование кода

1.7.1 Зачем нужны модули

1.7.2 Создание модуля для PostgreSQL Database

1.7.3 Использование модуля

1.7.4 Публикация модуля в Terraform Registry

1.8 State management: безопасная работа в команде

1.8.1 Что такое Terraform State

1.8.2 Проблемы локального State

1.8.3 Решение: Remote Backend

1.8.4 Настройка S3 Backend (AWS)

1.8.5 Работа в команде с Remote Backend

1.8.6 Разделение State по окружениям

1.8.7 Откат State к предыдущей версии

1.9 CI/CD для баз данных с Terraform

1.9.1 GitOps Workflow для PostgreSQL

1.9.2 GitHub Actions: Terraform CI/CD

1.9.3 Интеграция с миграциями баз данных

1.10 Практические рецепты и шаблоны

1.10.1 Рецепт 1: Автоматическое создание расширений для новых баз

1.10.2 Рецепт 2: Генерация случайных паролей с сохранением в AWS Secrets Manager

1.10.3 Рецепт 3: Автоматическая ротация паролей

1.10.4 Рецепт 4: Импорт существующих ресурсов в Terraform

1.10.5 Рецепт 5: Управление множеством баз через CSV

1.11 Troubleshooting и подводные камни

1.11.1 Проблема 1: “Error: pq: password authentication failed”

1.11.2 Проблема 2: “Error: role already exists”

1.11.3 Проблема 3: Drift Detection — ресурс изменён вручную

1.11.4 Проблема 4: State Lock — кто-то забыл разблокировать

1.11.5 Проблема 5: Циклические зависимости

1.11.6 Проблема 6: Terraform хочет пересоздать ресурс (- / +)

1.11.7 Проблема 7: “Error: extension does not exist”

1.11.8 Проблема 8: Secrets в логах Terraform

1.11.9 Проблема 9: Изменение пароля не применяется

1.12 Полезные ресурсы

1.12.1 Официальная документация

1.12.2 Книги

1.12.3 Блоги и статьи

1.12.4 Инструменты и утилиты

1.12.5 Сообщества

1.12.6 Курсы

1.13 Заключение

1.13.1 Что вы теперь знаете

1.13.2 Преимущества подхода IaC для PostgreSQL DBA

1.13.3 Что дальше

1 Terraform для PostgreSQL DBA: От ручного управления к Infrastructure as Code

Дополнительный материал к курсу PostgreSQL Advanced Автор: Дмитрий Золотов

1.1 Оглавление

1. Зачем DBA изучать Terraform
2. Infrastructure as Code: парадигма управления инфраструктурой
3. Terraform: основы для начинающих
4. Управление PostgreSQL в облаках
5. Terraform PostgreSQL Provider: управление внутри базы
6. Модули и переиспользование кода
7. State management: безопасная работа в команде
8. CI/CD для баз данных с Terraform
9. Практические рецепты и шаблоны
10. Troubleshooting и подводные камни
11. Полезные ресурсы

1.2 Зачем DBA изучать Terraform

1.2.1 Мотивация: что изменится в вашей работе

Вы — опытный PostgreSQL DBA. Возможно, вы управляете десятками инстансов PostgreSQL: создаёте пользователей, настраиваете роли, создаёте схемы, управляете расширениями. Каждое действие — это набор SQL-команд, которые вы выполняете вручную или через самописные скрипты.

Типичный день без IaC:

```
# На Production
psql -h prod-db.company.com -U postgres << 'EOF'
CREATE ROLE app_user WITH LOGIN PASSWORD 'secret123';
GRANT CONNECT ON DATABASE myapp TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO
        app_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE, DELETE
        ON TABLES TO app_user;
EOF

# Через неделю на Staging (а как там было на prod?)
psql -h staging-db.company.com -U postgres << 'EOF'
CREATE ROLE app_user WITH LOGIN PASSWORD 'staging_secret';
GRANT CONNECT ON DATABASE myapp TO app_user;
-- Упс, забыли USAGE ON SCHEMA
GRANT SELECT, INSERT ON ALL TABLES IN SCHEMA public TO app_user;
-- Упс, нет UPDATE и DELETE, а приложение падает...
EOF
```

Проблемы этого подхода:

- ❌ **Нет версионирования** — невозможно откатиться к предыдущему состоянию
- ❌ **Расхождение между окружениями** — prod, staging, dev отличаются
- ❌ **Человеческий фактор** — забыли команду, опечатка в SQL
- ❌ **Нет аудита** — кто, когда и что менял?
- ❌ **Сложность масштабирования** — 50 баз = 50 раз выполнить вручную
- ❌ **Документация устаревает** — wiki не отражает реальность

1.2.2 Что даёт Terraform

Infrastructure as Code (IaC) — это подход, при котором инфраструктура описывается в виде кода (декларативных конфигураций), который можно версионировать, тестировать и применять автоматически.

Тот же сценарий с Terraform:

```
# modules/database-user/main.tf
resource "postgresql_role" "app_user" {
  name     = "app_user"
  login    = true
  password = var.password
}

resource "postgresql_grant" "app_user_connect" {
  database    = "myapp"
  role        = postgresql_role.app_user.name
  object_type = "database"
  privileges  = ["CONNECT"]
}

resource "postgresql_grant" "app_user_schema" {
  database    = "myapp"
  role        = postgresql_role.app_user.name
  schema      = "public"
  object_type = "schema"
  privileges  = ["USAGE"]
}

resource "postgresql_default_privileges" "app_user_tables" {
  database    = "myapp"
  role        = postgresql_role.app_user.name
  schema      = "public"
  owner       = "postgres"
  object_type = "table"
  privileges  = ["SELECT", "INSERT", "UPDATE", "DELETE"]
}
```

Применение на всех окружениях:

```
# Production
terraform apply -var-file=prod.tfvars

# Staging
terraform apply -var-file=staging.tfvars

# Dev
terraform apply -var-file=dev.tfvars
```

Результат:

- ✅ **Одна спецификация** — все окружения идентичны
- ✅ **Версионирование** — Git хранит всю историю изменений
- ✅ **Code review** — каждое изменение проходит проверку
- ✅ **Откат** — `git revert` + `terraform apply`

- ✅ **Аудит** — кто, когда, что менял (Git log)
- ✅ **Масштабирование** — 1 команда = 50 баз обновлены

1.2.3 Реальные кейсы из индустрии

Zalando (E-commerce, Германия):

- Управляют 300+ PostgreSQL кластерами через Terraform
- Все изменения проходят через GitOps workflow
- Среднее время создания нового окружения: 5 минут (было: 2 дня)
- Zero инцидентов из-за человеческого фактора за последний год

Gitlab:

- 100% инфраструктуры PostgreSQL описано в Terraform
- Автоматическое тестирование миграций на staging перед prod
- Disaster recovery plan тестируется каждую неделю автоматически

Netflix:

- Управление сотнями баз данных через IaC
- Автоматическое создание read-реплик при росте нагрузки
- Self-service для разработчиков: новая база за 10 минут

1.2.4 Типичные боли DBA, которые решает Terraform

Проблема 1: “А как это было на проде?”

Вы настроили production 6 месяцев назад. Теперь нужно поднять staging. Как там было с правами? Какие расширения установлены? Какие параметры `postgresql.conf`?

Решение Terraform: Весь конфиг в Git. Просто посмотрите файл, созданный 6 месяцев назад.

Проблема 2: “Кто сломал права доступа?”

После обеда приложение перестало работать. Кто-то изменил права роли. Кто? Когда? Что именно?

Решение Terraform: Git log показывает commit, автора, дату. Можно сразу откатить: `git revert`.

Проблема 3: “Копипаста SQL-скриптов”

У вас 20 микросервисов. Для каждого нужна база, роль, схема. Вы копируете SQL-скрипт 20 раз, меняя имена. Где-то забыли, где-то опечатка.

Решение Terraform: Модуль описывает шаблон. Применяете 20 раз с разными параметрами. Гарантия консистентности.

Проблема 4: “Compliance и аудит”

Аудиторы требуют доказать, что все изменения авторизованы, задокументированы и прошли ревью.

Решение Terraform: Каждое изменение = Pull Request в Git. История review, approval, apply логируется.

1.3 Infrastructure as Code: парадигма управления инфраструктурой

1.3.1 Что такое IaC и почему это важно

Infrastructure as Code (IaC) — методология, при которой инфраструктура описывается в виде кода (конфигурационных файлов), который можно версионировать, тестировать и применять автоматически.

Основные принципы IaC:

1. **Декларативность** — вы описываете желаемое состояние, а не шаги для его достижения
2. **Идемпотентность** — повторное применение даёт тот же результат
3. **Версионирование** — весь код в системе контроля версий (Git)
4. **Автоматизация** — применение без ручного вмешательства
5. **Воспроизводимость** — из одного кода получается одна и та же инфраструктура

1.3.2 Сравнение подходов: Императивный vs Декларативный

Императивный подход (Shell-скрипты, Ansible playbooks с задачами):

```bash
#!/bin/bash
# create_user.sh

# Проверяем, существует ли роль
if psql -tAc "SELECT 1 FROM pg_roles WHERE rolname='app_user'" | grep -q 1;
        then
    echo "Role exists, updating password..."
    psql -c "ALTER ROLE app_user WITH PASSWORD 'new_password';"
else
    echo "Creating role..."
    psql -c "CREATE ROLE app_user WITH LOGIN PASSWORD 'new_password';"
fi

# Проверяем права на базу
if psql -tAc "SELECT 1 FROM information_schema.role_table_grants WHERE
        grantee='app_user' AND table_schema='public';" | grep -q 1; then
    echo "Grants already exist"
else
    echo "Granting privileges..."
    psql -c "GRANT SELECT, INSERT ON ALL TABLES IN SCHEMA public TO app_user;"
fi
```

Проблемы:

- Вы описываете **КАК** достичь состояния (алгоритм)
- Требуется логика проверки текущего состояния
- Сложно обрабатывать изменения (что если нужно отозвать INSERT?)
- Трудно понять текущее состояние системы из кода

Декларативный подход (Terraform):

```
# main.tf

resource "postgresql_role" "app_user" {
  name     = "app_user"
  login    = true
  password = "new_password"
}

resource "postgresql_grant" "app_user_tables" {
  database    = "myapp"
  role        = postgresql_role.app_user.name
  schema      = "public"
  object_type = "table"
  privileges  = ["SELECT", "INSERT"]
}
```

Преимущества:

- Вы описываете **ЧТО** должно быть (желаемое состояние)
- Terraform сам определяет, что создать, изменить или удалить
- Изменения очевидны: убрали INSERT из списка — Terraform отзовёт его
- Код = актуальная документация инфраструктуры

1.3.3 Аналогия для DBA: Репликация

Представьте, как работает streaming replication в PostgreSQL:

Master (Primary):

```
-- Вы делаете изменения
INSERT INTO users (name) VALUES ('Alice');
UPDATE users SET email = 'alice@example.com' WHERE name = 'Alice';
DELETE FROM orders WHERE id = 123;
```

Replica (Standby):

Реплика **не повторяет SQL-команды**. Она применяет WAL-записи, которые описывают **изменения состояния** на уровне блоков данных. Если вы выполните `INSERT` дважды, в WAL попадут две записи, но реплика просто применит обе.

Terraform работает похоже:

1. Вы описываете желаемое состояние (конфиг)
2. Terraform читает текущее состояние инфраструктуры (State)
3. Terraform вычисляет diff (план изменений)
4. Применяет минимальные изменения для достижения желаемого состояния

1.3.4 Типы IaC-инструментов

Таблица сравнения:

Инструмент	Тип	Область применения	Идемпотентность	Язык конфигурации
Terraform	Declarative	Мультиоблачная инфраструктура	✅ Да	HCL (HashiCorp Configuration Language)
Ansible	Imperative (может быть declarative)	Конфигурация серверов, оркестрация	⚠️ Частично (зависит от модулей)	YAML
CloudFormation	Declarative	AWS-инфраструктура	✅ Да	JSON / YAML
Pulumi	Declarative	Мультиоблачная инфраструктура	✅ Да	Python, TypeScript, Go, C#
Chef/Puppet	Imperative	Конфигурация серверов	⚠️ Частично	Ruby DSL / Puppet DSL

Почему Terraform популярен для DBA:

1. **Мультиоблачность** — один инструмент для AWS RDS, Azure Database, GCP CloudSQL
2. **Провайдер для PostgreSQL** — управление схемами, ролями, расширениями внутри БД
3. **Декларативность** — проще читать и понимать
4. **State management** — Terraform помнит, что создано, и следит за drift
5. **Большое сообщество** — тысячи готовых модулей

1.4 Terraform: основы для начинающих

1.4.1 Установка Terraform

MacOS:

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform

# Проверка установки
terraform version
# Terraform v1.7.0
```

Linux (Ubuntu/Debian):

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/
        share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
        https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo
        tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform
```

Windows:

```
choco install terraform

# Или скачайте бинарник с https://www.terraform.io/downloads
```

1.4.2 Структура проекта Terraform

Минимальный проект:

```
my-postgres-project/
├── main.tf              # Основные ресурсы
├── variables.tf         # Входные переменные
├── outputs.tf           # Выходные значения
├── terraform.tfvars     # Значения переменных (не коммитить!)
├── versions.tf          # Версии провайдеров
└── .gitignore           # Игнорируем State и секреты
```

Файл `.gitignore` (КРИТИЧЕСКИ ВАЖНО!):

```
# Terraform State (может содержать пароли!)
*.tfstate
*.tfstate.*

# Локальные переменные с секретами
terraform.tfvars
*.auto.tfvars

# Директория с плагинами провайдеров
.terraform/
.terraform.lock.hcl

# Crash logs
crash.log

# Планы (могут содержать секреты)
*.tfplan
```

1.4.3 Ваш первый Terraform-проект: Создание базы данных и роли

Шаг 1: Настройка провайдера PostgreSQL

Файл: `versions.tf`

```
terraform {
  required_version = ">= 1.7.0"

  required_providers {
    postgresql = {
      source  = "cyrilgdn/postgresql"
      version = "~> 1.21"
    }
  }
}
```

Файл: `main.tf`

```
# Провайдер для подключения к PostgreSQL
provider "postgresql" {
  host            = var.db_host
  port            = var.db_port
  database        = "postgres"  # Подключаемся к postgres для создания БД
  username        = var.db_admin_user
  password        = var.db_admin_password
  sslmode         = var.db_sslmode
  connect_timeout = 15
}

# Создание базы данных
resource "postgresql_database" "myapp" {
  name              = "myapp"
  owner             = "postgres"
  encoding          = "UTF8"
  lc_collate        = "en_US.UTF-8"
  lc_ctype          = "en_US.UTF-8"
  connection_limit  = 100
  allow_connections = true
}

# Создание роли для приложения
resource "postgresql_role" "app_user" {
  name     = "app_user"
  login    = true
  password = var.app_user_password

  # Настройки безопасности
  connection_limit = 20
  valid_until      = "infinity"

  # Права на уровне кластера
  superuser         = false
  create_database   = false
  create_role       = false
  inherit           = true
  replication       = false
  bypass_rls        = false
}

# Права на подключение к базе
resource "postgresql_grant" "app_user_database" {
  database    = postgresql_database.myapp.name
  role        = postgresql_role.app_user.name
  object_type = "database"
  privileges  = ["CONNECT", "TEMPORARY"]
}

# Права на схему public
resource "postgresql_grant" "app_user_schema" {
```

```
  database    = postgresql_database.myapp.name
  role        = postgresql_role.app_user.name
  schema      = "public"
  object_type = "schema"
  privileges  = ["USAGE"]
}

# Default privileges для будущих таблиц
resource "postgresql_default_privileges" "app_user_tables" {
  database    = postgresql_database.myapp.name
  role        = postgresql_role.app_user.name
  schema      = "public"
  owner       = "postgres"  # Кто будет создавать таблицы
  object_type = "table"
  privileges  = ["SELECT", "INSERT", "UPDATE", "DELETE"]
}

# Default privileges для последовательностей (sequences)
resource "postgresql_default_privileges" "app_user_sequences" {
  database    = postgresql_database.myapp.name
  role        = postgresql_role.app_user.name
  schema      = "public"
  owner       = "postgres"
  object_type = "sequence"
  privileges  = ["SELECT", "UPDATE", "USAGE"]
}
```

Файл: `variables.tf`

```
variable "db_host" {
  description = "PostgreSQL server hostname"
  type        = string
}

variable "db_port" {
  description = "PostgreSQL server port"
  type        = number
  default     = 5432
}

variable "db_admin_user" {
  description = "Admin username for PostgreSQL"
  type        = string
  default     = "postgres"
}

variable "db_admin_password" {
  description = "Admin password for PostgreSQL"
  type        = string
  sensitive   = true  # Terraform не покажет в логах
}

variable "db_sslmode" {
  description = "SSL mode for PostgreSQL connection"
  type        = string
  default     = "require"

  validation {
    condition     = contains(["disable", "require", "verify-ca", "verify-
full"], var.db_sslmode)
    error_message = "db_sslmode must be one of: disable, require, verify-ca,
verify-full"
  }
}

variable "app_user_password" {
  description = "Password for application user"
  type        = string
  sensitive   = true
}
```

Файл: `outputs.tf`

```
output "database_name" {
  description = "Name of the created database"
  value       = postgresql_database.myapp.name
}

output "app_user_name" {
  description = "Name of the application user"
  value       = postgresql_role.app_user.name
}

output "connection_string" {
  description = "Connection string for application (without password)"
  value       = "postgresql://${postgresql_role.app_user.name}@${var.db_host}:$
{var.db_port}/${postgresql_database.myapp.name}?sslmode=${var.db_sslmode}"
  sensitive   = true
}
```

Файл: `terraform.tfvars` (НЕ коммитить в Git!)

```
db_host            = "localhost"
db_port            = 5432
db_admin_user      = "postgres"
db_admin_password  = "your_admin_password_here"
db_sslmode         = "disable"  # Для локальной разработки
app_user_password  = "secure_app_password_123"
```

1.4.4 Workflow: Команды Terraform

1. Инициализация проекта

```
terraform init
```

Что происходит:

- Скачиваются плагины провайдеров (PostgreSQL provider)
- Инициализируется backend для хранения State
- Создаётся `.terraform/` директория

Вывод:

```
Initializing the backend...

Initializing provider plugins...
- Finding cyrilgdn/postgresql versions matching "~> 1.21"...
- Installing cyrilgdn/postgresql v1.21.0...
- Installed cyrilgdn/postgresql v1.21.0

Terraform has been successfully initialized!
```

2. Проверка конфигурации

```
terraform validate
```

Проверяет:

- Синтаксис HCL
- Корректность ссылок между ресурсами
- Обязательные параметры заполнены

3. Предпросмотр изменений (План)

```
terraform plan
```

Что показывает:

```
Terraform will perform the following actions:

  # postgresql_database.myapp will be created
  + resource "postgresql_database" "myapp" {
      + allow_connections = true
      + connection_limit  = 100
      + encoding          = "UTF8"
      + id                = (known after apply)
      + lc_collate        = "en_US.UTF-8"
      + lc_ctype          = "en_US.UTF-8"
      + name              = "myapp"
      + owner             = "postgres"
    }

  # postgresql_role.app_user will be created
  + resource "postgresql_role" "app_user" {
      + connection_limit = 20
      + id               = (known after apply)
      + login            = true
      + name             = "app_user"
      + password         = (sensitive value)
      + skip_drop_role   = false
      + superuser        = false
    }

  # postgresql_grant.app_user_database will be created
  + resource "postgresql_grant" "app_user_database" {
      + database    = "myapp"
      + id          = (known after apply)
      + object_type = "database"
      + privileges  = [
          + "CONNECT",
          + "TEMPORARY",
        ]
      + role        = "app_user"
    }

Plan: 5 to add, 0 to change, 0 to destroy.
```

Символы в плане:

- `+` — ресурс будет создан
- `~` — ресурс будет изменён (in-place или with replacement)
- `-/+` — ресурс будет пересоздан (destroy + create)
- `-` — ресурс будет удалён

4. Применение изменений

```
terraform apply
```

Terraform попросит подтверждение:

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```

Автоматическое применение (для CI/CD):

```
terraform apply -auto-approve
```

5. Просмотр текущего состояния

```
terraform show
```

Показывает созданные ресурсы и их атрибуты.

6. Вывод значений outputs

```
terraform output

# Конкретный output
terraform output connection_string
```

7. Уничтожение инфраструктуры

```
terraform destroy
```

⚠️ ОПАСНО! Удалит ВСЕ ресурсы, описанные в конфигурации.

1.4.5 Типичные ошибки новичков

Ошибка 1: Забыли `.gitignore`

```
# ❌ НИКОГДА НЕ КОММИТИТЬ!
git add terraform.tfstate
git commit -m "Added state"
```

Почему опасно:

- State файл содержит **пароли в открытом виде**
- State может содержать чувствительные данные (IP-адреса, структура инфраструктуры)

Решение:

```
# В .gitignore
*.tfstate
*.tfstate.*
terraform.tfvars
```

Ошибка 2: Hardcoded-пароли в коде

```
# ❌ НИКОГДА ТАК НЕ ДЕЛАЙТЕ!
resource "postgresql_role" "app_user" {
  name     = "app_user"
  password = "hardcoded_password_123"  # Попадёт в Git!
}
```

Решение:

```
# ✅ Правильно: переменная
resource "postgresql_role" "app_user" {
  name     = "app_user"
  password = var.app_user_password  # Значение из terraform.tfvars (не в Git)
}
```

Или используйте environment variables:

```
export TF_VAR_app_user_password="secure_password"
terraform apply
```

Ошибка 3: Изменение ресурса вручную

```
-- Руками в psql
ALTER ROLE app_user WITH CONNECTION LIMIT 50;
```

Terraform не знает об этом!

При следующем `terraform apply` он может вернуть лимит обратно к 20 (из конфига).

Решение:

- ✅ **Все изменения только через Terraform**

- Используйте `terraform import` для существующих ресурсов

Ошибка 4: Использование одного State для всех окружений

```
# ❌ Плохо: один State для prod и dev
terraform apply  # Может случайно изменить prod!
```

Решение: Workspaces или отдельные State файлы

```
# Создать workspace для staging
terraform workspace new staging
terraform workspace select staging
terraform apply

# Для production
terraform workspace new production
terraform workspace select production
terraform apply
```

1.5 Управление PostgreSQL в облаках

1.5.1 AWS RDS: Создание и управление PostgreSQL

Terraform AWS Provider:

```
# versions.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

Создание RDS PostgreSQL:

```
# main.tf

# Security Group для RDS
resource "aws_security_group" "postgresql_rds" {
  name_prefix = "postgresql-rds-"
  description = "Security group for PostgreSQL RDS"
  vpc_id      = var.vpc_id

  # Разрешить входящие подключения на порт 5432 только из VPC
  ingress {
    description = "PostgreSQL from VPC"
    from_port   = 5432
    to_port     = 5432
    protocol    = "tcp"
    cidr_blocks = [var.vpc_cidr]
  }

  # Исходящий трафик (для репликации, бэкапов и т.д.)
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name        = "postgresql-rds-sg"
    Environment = var.environment
  }
}

# DB Subnet Group (RDS должна быть в приватных подсетях)
resource "aws_db_subnet_group" "postgresql" {
  name_prefix = "postgresql-"
  description = "Subnet group for PostgreSQL RDS"
  subnet_ids  = var.private_subnet_ids

  tags = {
    Name        = "postgresql-subnet-group"
    Environment = var.environment
  }
}

# Parameter Group для настроек PostgreSQL
resource "aws_db_parameter_group" "postgresql16" {
  name_prefix = "postgres16-custom-"
  family      = "postgres16"  # Версия PostgreSQL
  description = "Custom parameter group for PostgreSQL 16"

  # Примеры настроек (адаптируйте под вашу нагрузку)
  parameter {
```

```
    name  = "shared_buffers"
    value = "{DBInstanceClassMemory/4096}"  # 25% от RAM
  }

  parameter {
    name  = "effective_cache_size"
    value = "{DBInstanceClassMemory*3/4096}"  # 75% от RAM
  }

  parameter {
    name  = "maintenance_work_mem"
    value = "2097152"  # 2 GB в KB
  }

  parameter {
    name  = "checkpoint_completion_target"
    value = "0.9"
  }

  parameter {
    name  = "wal_buffers"
    value = "16384"  # 16 MB
  }

  parameter {
    name  = "default_statistics_target"
    value = "100"
  }

  parameter {
    name  = "random_page_cost"
    value = "1.1"  # Для SSD
  }

  parameter {
    name  = "effective_io_concurrency"
    value = "200"  # Для SSD
  }

  parameter {
    name  = "work_mem"
    value = "10485"  # 10 MB в KB
  }

  parameter {
    name  = "min_wal_size"
    value = "1024"  # 1 GB в MB
  }

  parameter {
    name  = "max_wal_size"
    value = "4096"  # 4 GB в MB
```

```
  }

  # Логирование
  parameter {
    name  = "log_min_duration_statement"
    value = "1000"  # Логировать запросы > 1 сек
  }

  parameter {
    name  = "log_connections"
    value = "1"
  }

  parameter {
    name  = "log_disconnections"
    value = "1"
  }

  parameter {
    name  = "log_statement"
    value = "ddl"  # Логировать DDL
  }

  tags = {
    Name        = "postgresql16-custom"
    Environment = var.environment
  }
}

# RDS Instance
resource "aws_db_instance" "postgresql" {
  identifier_prefix = "myapp-postgres-"

  # Engine
  engine               = "postgres"
  engine_version       = "16.1"
  instance_class       = var.db_instance_class  # Например: db.t3.medium,
db.r6g.xlarge

  # Storage
  allocated_storage     = var.db_allocated_storage  # GB
  max_allocated_storage = var.db_max_allocated_storage  # Автоскейлинг
  storage_type          = "gp3"  # gp3 дешевле и быстрее gp2
  storage_encrypted     = true
  kms_key_id           = var.kms_key_arn  # Опционально: свой KMS key

  # Database
  db_name  = var.db_name
  username = var.db_master_username
  password = var.db_master_password  # Из переменной (чувствительная!)
  port     = 5432
```

```
  # Network
  db_subnet_group_name   = aws_db_subnet_group.postgresql.name
  vpc_security_group_ids = [aws_security_group.postgresql_rds.id]
  publicly_accessible    = false  # НИКОГДА не делайте true для prod!

  # Parameter Group
  parameter_group_name = aws_db_parameter_group.postgresql16.name

  # Backups
  backup_retention_period = var.backup_retention_days  # 7-35 для prod
  backup_window           = "03:00-04:00"  # UTC
  maintenance_window      = "sun:04:00-sun:05:00"  # UTC

  # High Availability
  multi_az               = var.enable_multi_az  # true для prod

  # Monitoring
  enabled_cloudwatch_logs_exports = ["postgresql", "upgrade"]
  monitoring_interval             = 60  # Enhanced Monitoring (секунды)
  monitoring_role_arn             = aws_iam_role.rds_monitoring.arn
  performance_insights_enabled    = true
  performance_insights_retention_period = 7  # Дней

  # Deletion Protection
  deletion_protection = var.enable_deletion_protection  # true для prod
  skip_final_snapshot = var.environment == "dev" ? true : false
  final_snapshot_identifier = var.environment == "dev" ? null : "$
{var.db_name}-final-snapshot-${formatdate("YYYY-MM-DD-hhmm", timestamp())}"

  # Apply changes immediately (для dev) или во время maintenance window (для
prod)
  apply_immediately = var.environment == "dev" ? true : false

  tags = {
    Name        = "myapp-postgresql"
    Environment = var.environment
  }

  lifecycle {
    # Предотвращаем случайное пересоздание базы при изменении snapshot
identifier
    ignore_changes = [final_snapshot_identifier]
  }
}

# IAM Role для Enhanced Monitoring
resource "aws_iam_role" "rds_monitoring" {
  name_prefix = "rds-monitoring-"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
```

```
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "monitoring.rds.amazonaws.com"
        }
      }
    ]
  })
}

resource "aws_iam_role_policy_attachment" "rds_monitoring" {
  role       = aws_iam_role.rds_monitoring.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/
AmazonRDSEnhancedMonitoringRole"
}

# Read Replica (опционально)
resource "aws_db_instance" "postgresql_replica" {
  count = var.enable_read_replica ? 1 : 0

  identifier_prefix  = "myapp-postgres-replica-"
  replicate_source_db = aws_db_instance.postgresql.identifier

  instance_class     = var.replica_instance_class
  publicly_accessible = false

  # Read Replica не имеет собственных бэкапов
  backup_retention_period = 0

  # Мониторинг
  monitoring_interval    = 60
  monitoring_role_arn    = aws_iam_role.rds_monitoring.arn
  performance_insights_enabled = true

  tags = {
    Name        = "myapp-postgresql-replica"
    Environment = var.environment
  }
}
```

Переменные:

```
# variables.tf

variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "environment" {
  description = "Environment (dev, staging, prod)"
  type        = string

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod"
  }
}

variable "vpc_id" {
  description = "VPC ID where RDS will be deployed"
  type        = string
}

variable "vpc_cidr" {
  description = "VPC CIDR block for security group rules"
  type        = string
}

variable "private_subnet_ids" {
  description = "List of private subnet IDs for RDS"
  type        = list(string)
}

variable "db_instance_class" {
  description = "RDS instance class"
  type        = string
  default     = "db.t3.medium"
}

variable "db_allocated_storage" {
  description = "Allocated storage in GB"
  type        = number
  default     = 100
}

variable "db_max_allocated_storage" {
  description = "Maximum storage for autoscaling in GB"
  type        = number
  default     = 500
}
```

```
variable "db_name" {
  description = "Initial database name"
  type        = string
  default     = "myapp"
}

variable "db_master_username" {
  description = "Master username"
  type        = string
  default     = "postgres"
}

variable "db_master_password" {
  description = "Master password"
  type        = string
  sensitive   = true
}

variable "backup_retention_days" {
  description = "Backup retention period in days"
  type        = number
  default     = 7
}

variable "enable_multi_az" {
  description = "Enable Multi-AZ deployment"
  type        = bool
  default     = false
}

variable "enable_deletion_protection" {
  description = "Enable deletion protection"
  type        = bool
  default     = false
}

variable "enable_read_replica" {
  description = "Create a read replica"
  type        = bool
  default     = false
}

variable "replica_instance_class" {
  description = "Instance class for read replica"
  type        = string
  default     = "db.t3.medium"
}

variable "kms_key_arn" {
  description = "KMS key ARN for encryption (optional)"
  type        = string
```

```
  default     = null
}
```

Outputs:

```
# outputs.tf

output "db_instance_endpoint" {
  description = "RDS instance endpoint"
  value       = aws_db_instance.postgresql.endpoint
}

output "db_instance_address" {
  description = "RDS instance address (hostname)"
  value       = aws_db_instance.postgresql.address
}

output "db_instance_arn" {
  description = "ARN of the RDS instance"
  value       = aws_db_instance.postgresql.arn
}

output "db_instance_id" {
  description = "RDS instance identifier"
  value       = aws_db_instance.postgresql.id
}

output "read_replica_endpoint" {
  description = "Read replica endpoint (if enabled)"
  value       = var.enable_read_replica ?
aws_db_instance.postgresql_replica[0].endpoint : null
}

output "connection_string" {
  description = "PostgreSQL connection string (without password)"
  value       = "postgresql://${var.db_master_username}@$
{aws_db_instance.postgresql.endpoint}/${var.db_name}?sslmode=require"
  sensitive   = true
}
```

Применение:

```
# dev.tfvars
environment                 = "dev"
vpc_id                      = "vpc-0123456789abcdef0"
vpc_cidr                    = "10.0.0.0/16"
private_subnet_ids          = ["subnet-111", "subnet-222"]
db_instance_class           = "db.t3.medium"
db_allocated_storage        = 50
db_max_allocated_storage    = 200
db_master_password          = "DevPassword123!"
backup_retention_days       = 3
enable_multi_az             = false
enable_deletion_protection = false
enable_read_replica         = false

# prod.tfvars
environment                 = "prod"
vpc_id                      = "vpc-prod123456"
vpc_cidr                    = "10.1.0.0/16"
private_subnet_ids          = ["subnet-prod111", "subnet-prod222", "subnet-
        prod333"]
db_instance_class           = "db.r6g.2xlarge"  # 8 vCPU, 64 GB RAM
db_allocated_storage        = 500
db_max_allocated_storage    = 2000
db_master_password          = "SuperSecureProductionPassword!"  # Лучше из
        Secrets Manager
backup_retention_days       = 30
enable_multi_az             = true  # High Availability
enable_deletion_protection = true  # Защита от случайного удаления
enable_read_replica         = true  # Read scaling
```

Команды:

```
# Инициализация
terraform init

# Проверка
terraform validate

# План для dev
terraform plan -var-file=dev.tfvars

# Применение
terraform apply -var-file=dev.tfvars

# После применения - получить endpoint
terraform output db_instance_endpoint
# myapp-postgres-20240101123456.abcdef.us-east-1.rds.amazonaws.com:5432

# Подключиться к созданной RDS
psql "postgresql://postgres:DevPassword123!@myapp-
        postgres-20240101123456.abcdef.us-east-1.rds.amazonaws.com:5432/myapp?
        sslmode=require"
```

1.5.2 Azure Database for PostgreSQL

Аналогичный пример для Azure:

```
# versions.tf
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

# main.tf

# Resource Group
resource "azurerm_resource_group" "postgres" {
  name     = "rg-postgres-${var.environment}"
  location = var.azure_region
}

# PostgreSQL Flexible Server
resource "azurerm_postgresql_flexible_server" "main" {
  name                = "psql-${var.app_name}-${var.environment}"
  resource_group_name = azurerm_resource_group.postgres.name
  location            = azurerm_resource_group.postgres.location

  version             = "16"
  administrator_login = var.db_admin_username
  administrator_password = var.db_admin_password

  sku_name   = var.db_sku_name  # Например: GP_Standard_D2ds_v4
  storage_mb = var.db_storage_mb

  backup_retention_days = var.backup_retention_days
  geo_redundant_backup_enabled = var.enable_geo_redundant_backup

  high_availability {
    mode                      = var.enable_ha ? "ZoneRedundant" : "Disabled"
    standby_availability_zone = var.enable_ha ? var.standby_zone : null
  }

  tags = {
    Environment = var.environment
  }
}

# Database
resource "azurerm_postgresql_flexible_server_database" "myapp" {
  name      = var.db_name
```

```
  server_id = azurerm_postgresql_flexible_server.main.id
  charset   = "UTF8"
  collation = "en_US.utf8"
}

# Firewall Rule (для доступа из Azure Services)
resource "azurerm_postgresql_flexible_server_firewall_rule" "azure_services" {
  name             = "AllowAzureServices"
  server_id        = azurerm_postgresql_flexible_server.main.id
  start_ip_address = "0.0.0.0"
  end_ip_address   = "0.0.0.0"
}

# Configuration (параметры PostgreSQL)
resource "azurerm_postgresql_flexible_server_configuration" "shared_buffers" {
  name      = "shared_buffers"
  server_id = azurerm_postgresql_flexible_server.main.id
  value     = "262144"  # 2 GB (в 8KB блоках)
}

resource "azurerm_postgresql_flexible_server_configuration" "work_mem" {
  name      = "work_mem"
  server_id = azurerm_postgresql_flexible_server.main.id
  value     = "10240"  # 10 MB в KB
}
```

Output:

```
output "fqdn" {
  value = azurerm_postgresql_flexible_server.main.fqdn
}

output "connection_string" {
  value     = "postgresql://${var.db_admin_username}@$
{azurerm_postgresql_flexible_server.main.fqdn}:5432/${var.db_name}?
sslmode=require"
  sensitive = true
}
```

1.6 Terraform PostgreSQL Provider: управление внутри базы

1.6.1 Что умеет PostgreSQL Provider

Terraform PostgreSQL Provider позволяет управлять объектами **внутри** существующей базы данных:

Объекты, которыми можно управлять:

- ✅ Базы данных (`postgresql_database`)
- ✅ Роли и пользователи (`postgresql_role`)
- ✅ Схемы (`postgresql_schema`)
- ✅ Расширения (`postgresql_extension`)
- ✅ Права доступа (`postgresql_grant`, `postgresql_default_privileges`)
- ✅ Replication Slots (`postgresql_replication_slot`)
- ✅ Publications/Subscriptions для логической репликации
- ✅ Functions, Procedures (через `postgresql_function`)

⚠️ Что НЕ умеет:

- ❌ Управление самим сервером PostgreSQL (используйте AWS/Azure/GCP провайдеры)
- ❌ Изменение `postgresql.conf` (используйте Parameter Groups в RDS или config management)
- ❌ Управление физической репликацией (это уровень сервера)

1.6.2 Управление схемами

Типичный сценарий: Мультитенантное приложение

У вас есть SaaS-приложение, где каждый клиент (tenant) имеет свою схему в одной базе данных.

```
# tenants.tf

locals {
  tenants = ["acme_corp", "widgets_inc", "foo_bar_llc"]
}

# Создание схемы для каждого клиента
resource "postgresql_schema" "tenant" {
  for_each = toset(local.tenants)

  name     = each.value
  owner    = "postgres"

  # При удалении схемы из Terraform, не удалять её из БД (защита)
  drop_cascade = false
}

# Создание роли для каждого клиента
resource "postgresql_role" "tenant" {
  for_each = toset(local.tenants)

  name     = "${each.value}_role"
  login    = true
  password = random_password.tenant[each.value].result

  connection_limit = 10
}

# Генерация паролей для клиентов
resource "random_password" "tenant" {
  for_each = toset(local.tenants)

  length  = 32
  special = true
}

# Права на схему для каждого клиента (только своя схема)
resource "postgresql_grant" "tenant_schema" {
  for_each = toset(local.tenants)

  database    = "myapp"
  role        = postgresql_role.tenant[each.value].name
  schema      = postgresql_schema.tenant[each.value].name
  object_type = "schema"
  privileges  = ["CREATE", "USAGE"]
}

# Default privileges: все таблицы в схеме
resource "postgresql_default_privileges" "tenant_tables" {
  for_each = toset(local.tenants)
```

```
  database    = "myapp"
  role        = postgresql_role.tenant[each.value].name
  schema      = postgresql_schema.tenant[each.value].name
  owner       = "postgres"
  object_type = "table"
  privileges  = ["SELECT", "INSERT", "UPDATE", "DELETE"]
}

# Output: пароли клиентов (сохраните в Secrets Manager!)
output "tenant_credentials" {
  value = {
    for tenant in local.tenants :
    tenant => {
      username = postgresql_role.tenant[tenant].name
      password = random_password.tenant[tenant].result
    }
  }
  sensitive = true
}
```

Применение:

```
terraform apply

# Получить пароли
terraform output -json tenant_credentials | jq

# Результат:
{
  "acme_corp": {
    "username": "acme_corp_role",
    "password": "xKf9#mP2..."
  },
  "widgets_inc": {
    "username": "widgets_inc_role",
    "password": "Qz7&nL5..."
  },
  ...
}
```

1.6.3 Управление расширениями

```
# extensions.tf

# PostGIS для геоданных
resource "postgresql_extension" "postgis" {
  name     = "postgis"
  schema   = "public"
  version  = "3.4.0"
  database = "myapp"
}

# pg_stat_statements для мониторинга запросов
resource "postgresql_extension" "pg_stat_statements" {
  name     = "pg_stat_statements"
  database = "postgres"  # Обычно устанавливается в postgres DB
}

# pgcrypto для шифрования
resource "postgresql_extension" "pgcrypto" {
  name     = "pgcrypto"
  schema   = "public"
  database = "myapp"
}

# pg_trgm для полнотекстового поиска
resource "postgresql_extension" "pg_trgm" {
  name     = "pg_trgm"
  schema   = "public"
  database = "myapp"
}

# Пример: установка расширения только если переменная enabled
resource "postgresql_extension" "timescaledb" {
  count = var.enable_timescaledb ? 1 : 0

  name     = "timescaledb"
  database = "myapp"

  # TimescaleDB требует специальных настроек shared_preload_libraries
  # Убедитесь, что в Parameter Group добавлено: shared_preload_libraries =
'timescaledb'
}
```

1.6.4 Сложный пример: Row-Level Security (RLS)

Terraform не имеет встроенного ресурса для RLS, но можно использовать `postgresql_function` или `null_resource` с `local-exec`:

```
# rls.tf

# Включение RLS на таблице через SQL
resource "null_resource" "enable_rls_on_users" {
  triggers = {
    # Перезапускать при изменении таблицы
    table_name = "users"
  }

  provisioner "local-exec" {
    command = <<-EOT
      psql "${var.database_url}" <<SQL
        ALTER TABLE users ENABLE ROW LEVEL SECURITY;

        CREATE POLICY tenant_isolation ON users
          USING (tenant_id = current_setting('app.current_tenant')::INT);

        GRANT SELECT, INSERT, UPDATE, DELETE ON users TO app_user;
      SQL
    EOT
  }

  depends_on = [
    postgresql_role.app_user,
    # Предполагается, что таблица users уже создана миграциями
  ]
}
```

⚠️ **Примечание:** `null_resource` с `local-exec` не идемпотентен. Для production используйте специализированные инструменты миграций (Liquibase, Flyway, Alembic).

1.7 Модули и переиспользование кода

1.7.1 Зачем нужны модули

Проблема без модулей:

У вас 10 микросервисов. Для каждого нужна:

- База данных
- Роль для приложения
- Роль для read-only доступа
- Схема
- Расширения (pg_stat_statements, pgcrypto)

Копируете один и тот же код 10 раз. Нашли баг? Исправляете в 10 местах.

Решение: Модули

Модуль — это переиспользуемый блок Terraform-кода с параметрами (как функция в программировании).

1.7.2 Создание модуля для PostgreSQL Database

Структура:

```
terraform-modules/
└── postgresql-database/
    ├── main.tf
    ├── variables.tf
    ├── outputs.tf
    └── README.md
```

Файл: `terraform-modules/postgresql-database/main.tf`

```
# Создание базы данных
resource "postgresql_database" "this" {
  name              = var.database_name
  owner             = var.database_owner
  encoding          = var.encoding
  lc_collate        = var.lc_collate
  lc_ctype          = var.lc_ctype
  connection_limit  = var.connection_limit
  allow_connections = var.allow_connections
}

# Схема (опционально)
resource "postgresql_schema" "this" {
  count = var.create_schema ? 1 : 0

  name     = var.schema_name != null ? var.schema_name : var.database_name
  database = postgresql_database.this.name
  owner    = var.database_owner
}

# Роль для приложения
resource "postgresql_role" "app" {
  name     = "${var.database_name}_app"
  login    = true
  password = var.app_password

  connection_limit = var.app_connection_limit
  superuser        = false
  create_database  = false
  create_role      = false
  replication      = false
}

# Роль для read-only
resource "postgresql_role" "readonly" {
  count = var.create_readonly_role ? 1 : 0

  name     = "${var.database_name}_readonly"
  login    = true
  password = var.readonly_password

  connection_limit = var.readonly_connection_limit
  superuser        = false
  create_database  = false
  create_role      = false
  replication      = false
}

# Права для приложения
resource "postgresql_grant" "app_database" {
  database    = postgresql_database.this.name
```

```
  role        = postgresql_role.app.name
  object_type = "database"
  privileges  = ["CONNECT", "TEMPORARY"]
}

resource "postgresql_grant" "app_schema" {
  count = var.create_schema ? 1 : 0

  database    = postgresql_database.this.name
  role        = postgresql_role.app.name
  schema      = postgresql_schema.this[0].name
  object_type = "schema"
  privileges  = ["USAGE", "CREATE"]
}

resource "postgresql_default_privileges" "app_tables" {
  count = var.create_schema ? 1 : 0

  database    = postgresql_database.this.name
  role        = postgresql_role.app.name
  schema      = postgresql_schema.this[0].name
  owner       = var.database_owner
  object_type = "table"
  privileges  = ["SELECT", "INSERT", "UPDATE", "DELETE"]
}

# Права для read-only
resource "postgresql_grant" "readonly_database" {
  count = var.create_readonly_role ? 1 : 0

  database    = postgresql_database.this.name
  role        = postgresql_role.readonly[0].name
  object_type = "database"
  privileges  = ["CONNECT"]
}

resource "postgresql_grant" "readonly_schema" {
  count = var.create_readonly_role && var.create_schema ? 1 : 0

  database    = postgresql_database.this.name
  role        = postgresql_role.readonly[0].name
  schema      = postgresql_schema.this[0].name
  object_type = "schema"
  privileges  = ["USAGE"]
}

resource "postgresql_default_privileges" "readonly_tables" {
  count = var.create_readonly_role && var.create_schema ? 1 : 0

  database    = postgresql_database.this.name
  role        = postgresql_role.readonly[0].name
  schema      = postgresql_schema.this[0].name
```

```
  owner       = var.database_owner
  object_type = "table"
  privileges  = ["SELECT"]
}

# Расширения
resource "postgresql_extension" "extensions" {
  for_each = toset(var.extensions)

  name     = each.value
  database = postgresql_database.this.name
  schema   = var.create_schema ? postgresql_schema.this[0].name : "public"
}
```

Файл: `terraform-modules/postgresql-database/variables.tf`

```
variable "database_name" {
  description = "Name of the database"
  type        = string
}

variable "database_owner" {
  description = "Owner of the database"
  type        = string
  default     = "postgres"
}

variable "encoding" {
  description = "Character encoding"
  type        = string
  default     = "UTF8"
}

variable "lc_collate" {
  description = "Collation order"
  type        = string
  default     = "en_US.UTF-8"
}

variable "lc_ctype" {
  description = "Character classification"
  type        = string
  default     = "en_US.UTF-8"
}

variable "connection_limit" {
  description = "Max connections to database (-1 = unlimited)"
  type        = number
  default     = -1
}

variable "allow_connections" {
  description = "Allow connections to this database"
  type        = bool
  default     = true
}

variable "create_schema" {
  description = "Create a schema within the database"
  type        = bool
  default     = true
}

variable "schema_name" {
  description = "Name of the schema (defaults to database_name)"
  type        = string
  default     = null
```

```
}

variable "app_password" {
  description = "Password for application role"
  type        = string
  sensitive   = true
}

variable "app_connection_limit" {
  description = "Connection limit for app role"
  type        = number
  default     = 20
}

variable "create_readonly_role" {
  description = "Create a read-only role"
  type        = bool
  default     = false
}

variable "readonly_password" {
  description = "Password for read-only role"
  type        = string
  sensitive   = true
  default     = null
}

variable "readonly_connection_limit" {
  description = "Connection limit for readonly role"
  type        = number
  default     = 10
}

variable "extensions" {
  description = "List of PostgreSQL extensions to enable"
  type        = list(string)
  default     = []
}
```

Файл: `terraform-modules/postgresql-database/outputs.tf`

```
output "database_name" {
  description = "Name of the created database"
  value       = postgresql_database.this.name
}

output "app_role_name" {
  description = "Name of the application role"
  value       = postgresql_role.app.name
}

output "readonly_role_name" {
  description = "Name of the read-only role (if created)"
  value       = var.create_readonly_role ? postgresql_role.readonly[0].name :
null
}

output "schema_name" {
  description = "Name of the created schema (if created)"
  value       = var.create_schema ? postgresql_schema.this[0].name : null
}
```

1.7.3 Использование модуля

Главный проект:

```
# main.tf

module "user_service_db" {
  source = "./terraform-modules/postgresql-database"

  database_name          = "user_service"
  app_password           = var.user_service_password
  create_readonly_role   = true
  readonly_password      = var.user_service_readonly_password
  extensions             = ["pgcrypto", "pg_trgm"]
  app_connection_limit   = 50
}

module "order_service_db" {
  source = "./terraform-modules/postgresql-database"

  database_name          = "order_service"
  app_password           = var.order_service_password
  create_readonly_role   = false
  extensions             = ["pg_stat_statements"]
  app_connection_limit   = 30
}

module "analytics_db" {
  source = "./terraform-modules/postgresql-database"

  database_name          = "analytics"
  app_password           = var.analytics_password
  create_readonly_role   = true
  readonly_password      = var.analytics_readonly_password
  extensions             = ["timescaledb", "pg_stat_statements"]
  connection_limit       = 100
  app_connection_limit   = 20
  readonly_connection_limit = 50
}
```

Применение:

```
terraform init
terraform plan
terraform apply
```

Результат: 3 базы, 4 роли (3 app + 2 readonly), схемы, расширения — всё создано из одного модуля.

1.7.4 Публикация модуля в Terraform Registry

Если модуль полезен сообществу, можно опубликовать его в Terraform Registry.

Требования:

- Модуль в отдельном Git-репозитории
- Структура: `main.tf`, `variables.tf`, `outputs.tf`, `README.md`, `LICENSE`
- Версионирование через Git tags (`v1.0.0`, `v1.1.0` и т.д.)
- GitHub/GitLab repo

Использование публичного модуля:

```
module "rds_postgres" {
  source  = "terraform-aws-modules/rds/aws"
  version = "~> 6.0"

  identifier = "myapp-postgres"
  engine     = "postgres"
  engine_version = "16.1"
  # ...
}
```

1.8 State management: безопасная работа в команде

1.8.1 Что такое Terraform State

State файл (`terraform.tfstate`) — это JSON-файл, который содержит:

- Какие ресурсы созданы Terraform
- Их текущее состояние (ID, атрибуты)
- Метаданные (dependencies, metadata)
- **ПАРОЛИ И СЕКРЕТЫ в открытом виде!**

Пример State:

```
{
  "version": 4,
  "terraform_version": "1.7.0",
  "serial": 5,
  "lineage": "abc123...",
  "resources": [
    {
      "type": "postgresql_role",
      "name": "app_user",
      "instances": [
        {
          "attributes": {
            "id": "app_user",
            "name": "app_user",
            "password": "plaintext_password_here",  // ⚠️ Секрет!
            "login": true
          }
        }
      ]
    }
  ]
}
```

1.8.2 Проблемы локального State

Проблема 1: Работа в команде

- Андрей применил изменения локально → его State обновился
- Борис не знает об изменениях → его State устарел
- Борис запускает `terraform apply` → конфликт, возможна потеря данных

Проблема 2: Безопасность

- State содержит пароли
- Локальный файл → риск случайного commit в Git
- Нет аудита: кто и когда менял State

Проблема 3: Disaster Recovery

- Ноутбук Андрея умер вместе с State
- Невозможно управлять инфраструктурой (Terraform не знает, что создано)

1.8.3 Решение: Remote Backend

Remote Backend — хранение State в удалённом защищённом хранилище с блокировками (locking) для предотвращения одновременных изменений.

Популярные backend:

Backend	Хранилище	Locking	Шифрование	Версионирование
S3 + DynamoDB	AWS S3	DynamoDB	AES-256 (SSE)	Да (versioning)
Azure Blob	Azure Storage	Azure Blob Lease	AES-256	Да
GCS	Google Cloud Storage	GCS	AES-256	Да
Terraform Cloud	HashiCorp Cloud	Встроенный	Да	Да
Consul	HashiCorp Consul	Consul	TLS	Нет
PostgreSQL	PostgreSQL таблица	Advisory Locks	На уровне БД	Нет

1.8.4 Настройка S3 Backend (AWS)

Шаг 1: Создать S3 bucket и DynamoDB таблицу для locking

```
# backend-setup.tf (запустить один раз для создания backend инфраструктуры)

terraform {
  # Пока используем local backend для создания remote backend (курица-яйцо
проблема)
  backend "local" {}
}

provider "aws" {
  region = "us-east-1"
}

# S3 bucket для State
resource "aws_s3_bucket" "terraform_state" {
  bucket = "mycompany-terraform-state"

  # Защита от случайного удаления
  lifecycle {
    prevent_destroy = true
  }

  tags = {
    Name = "Terraform State"
  }
}

# Включить версионирование (для отката State)
resource "aws_s3_bucket_versioning" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
}

# Шифрование State
resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state"
{
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

# Блокировать публичный доступ
resource "aws_s3_bucket_public_access_block" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id
```

```
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# DynamoDB таблица для State locking
resource "aws_dynamodb_table" "terraform_locks" {
  name         = "terraform-state-locks"
  billing_mode = "PAY_PER_REQUEST"  # Или PROVISIONED с read_capacity/
write_capacity
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }

  tags = {
    Name = "Terraform State Locks"
  }
}
```

Применить:

```
# Создать S3 bucket и DynamoDB таблицу
terraform init
terraform apply

# После создания:
# Bucket: mycompany-terraform-state
# Table: terraform-state-locks
```

Шаг 2: Настроить backend в основном проекте

```
# backend.tf (в основном проекте)

terraform {
  backend "s3" {
    bucket         = "mycompany-terraform-state"
    key            = "postgresql/prod/terraform.tfstate"  # Путь к State файлу
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-locks"

    # Опционально: использовать IAM role для доступа
    # role_arn = "arn:aws:iam::123456789012:role/TerraformStateRole"
  }
}
```

Мигрировать локальный State в S3:

```
# Terraform обнаружит изменение backend и предложит миграцию
terraform init

# Вывод:
# Initializing the backend...
# Do you want to copy existing state to the new backend?
#   Enter a value: yes

# Successfully configured the backend "s3"!
```

Проверить:

```
# State теперь в S3
aws s3 ls s3://mycompany-terraform-state/postgresql/prod/
# terraform.tfstate

# Lock-таблица в DynamoDB (пустая, если никто не работает)
aws dynamodb scan --table-name terraform-state-locks
```

1.8.5 Работа в команде с Remote Backend

Сценарий: два инженера работают одновременно

Инженер 1 (Андрей):

```
terraform apply
# Acquiring state lock. This may take a few moments...
# ...
```

Во время работы Андрея, Инженер 2 (Борис):

```
terraform apply
# Error: Error acquiring the state lock
#
# Lock Info:
#   ID:        abc123-456def-789ghi
#   Path:      mycompany-terraform-state/postgresql/prod/terraform.tfstate
#   Operation: OperationTypeApply
#   Who:       andrey@hostname
#   Created:   2026-02-13 10:30:00 UTC
#
# Terraform acquires a state lock to protect the state from being written
# by multiple users at the same time. Please resolve the issue above and try
# again. For most commands, you can disable locking with the "-lock=false"
# flag, but this is not recommended.
```

Борис ждёт, пока Андрей закончит.

После завершения Андрея:

```
# Борис может продолжить
terraform apply
# Acquiring state lock...
# ✅ Успешно!
```

1.8.6 Разделение State по окружениям

Плохо: один State для всех окружений

```
mycompany-terraform-state/
└── terraform.tfstate  # И dev, и prod здесь — опасно!
```

Хорошо: отдельные State для окружений

Вариант 1: Разные ключи (keys) в одном bucket

```
# backend-dev.tf
terraform {
  backend "s3" {
    bucket = "mycompany-terraform-state"
    key    = "postgresql/dev/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    dynamodb_table = "terraform-state-locks"
  }
}

# backend-prod.tf
terraform {
  backend "s3" {
    bucket = "mycompany-terraform-state"
    key    = "postgresql/prod/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    dynamodb_table = "terraform-state-locks"
  }
}
```

Вариант 2: Workspaces (для простых случаев)

```
# Создать workspace для dev
terraform workspace new dev

# Переключиться на prod
terraform workspace new prod
terraform workspace select prod

# Текущий workspace
terraform workspace show
# prod

# Список workspaces
terraform workspace list
#   default
#   dev
# * prod
```

State будет храниться в:

```
mycompany-terraform-state/
└── env:/
    ├── dev/
    |   └── terraform.tfstate
    └── prod/
        └── terraform.tfstate
```

Вариант 3: Отдельные директории проектов

```
terraform-projects/
├── dev/
|   ├── main.tf
|   ├── backend.tf  # key = "postgresql/dev/terraform.tfstate"
|   └── dev.tfvars
└── prod/
    ├── main.tf
    ├── backend.tf  # key = "postgresql/prod/terraform.tfstate"
    └── prod.tfvars
```

1.8.7 Откат State к предыдущей версии

Сценарий: случайно удалили ресурс

```
# Упс! Случайно применили terraform destroy
terraform destroy -auto-approve

# Нужно откатить State
```

Восстановление из S3 Versioning:

```
# 1. Список версий State
aws s3api list-object-versions \
  --bucket mycompany-terraform-state \
  --prefix postgresql/prod/terraform.tfstate

# Вывод:
# {
#   "Versions": [
#     {
#       "VersionId": "current_version_id",
#       "LastModified": "2026-02-13T11:00:00Z"
#     },
#     {
#       "VersionId": "previous_version_id",  # ← Нужна эта!
#       "LastModified": "2026-02-13T10:00:00Z"
#     }
#   ]
# }

# 2. Скачать предыдущую версию
aws s3api get-object \
  --bucket mycompany-terraform-state \
  --key postgresql/prod/terraform.tfstate \
  --version-id previous_version_id \
  terraform.tfstate.backup

# 3. Загрузить обратно (перезаписать текущую)
aws s3 cp terraform.tfstate.backup s3://mycompany-terraform-state/postgresql/
        prod/terraform.tfstate

# 4. Terraform снова видит ресурсы!
terraform plan
# No changes. Infrastructure is up-to-date.
```

1.9 CI/CD для баз данных с Terraform

1.9.1 GitOps Workflow для PostgreSQL

Идея: Все изменения инфраструктуры PostgreSQL проходят через Git и автоматически применяются через CI/CD pipeline.

Workflow:

1. DBA создаёт Pull Request с изменениями в Terraform
2. CI запускает `terraform plan` и показывает, что изменится

3. Команда делает code review
4. После approve и merge — автоматический `terraform apply` на staging
5. После тестирования — manual approval для prod
6. Автоматический `terraform apply` на prod

1.9.2 GitHub Actions: Terraform CI/CD

.github/workflows/terraform.yml:

```yaml
name: Terraform PostgreSQL CI/CD

on:
  pull_request:
    paths:
      - 'terraform/**'
  push:
    branches:
      - main
    paths:
      - 'terraform/**'

env:
  TF_VERSION: '1.7.0'
  AWS_REGION: 'us-east-1'

jobs:
  # Job 1: Проверка и планирование (на каждый PR)
  terraform-plan:
    name: Terraform Plan
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ env.TF_VERSION }}

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Terraform Init
        working-directory: ./terraform
        run: terraform init

      - name: Terraform Format Check
        working-directory: ./terraform
        run: terraform fmt -check -recursive

      - name: Terraform Validate
        working-directory: ./terraform
        run: terraform validate
```

```yaml
      - name: Terraform Plan (Dev)
        working-directory: ./terraform
        env:
          TF_VAR_db_admin_password: ${{ secrets.DEV_DB_ADMIN_PASSWORD }}
          TF_VAR_app_user_password: ${{ secrets.DEV_APP_PASSWORD }}
        run: |
          terraform workspace select dev || terraform workspace new dev
          terraform plan -var-file=dev.tfvars -no-color -out=tfplan

      - name: Save Plan
        uses: actions/upload-artifact@v4
        with:
          name: terraform-plan
          path: terraform/tfplan

      - name: Comment PR with Plan
        uses: actions/github-script@v7
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          script: |
            const fs = require('fs');
            const plan = fs.readFileSync('terraform/tfplan.txt', 'utf8');

            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: `## Terraform Plan\n\`\`\`\n${plan}\n\`\`\``
            });

  # Job 2: Применение на Dev (автоматически после merge в main)
  terraform-apply-dev:
    name: Apply to Dev
    runs-on: ubuntu-latest
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    environment: dev

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ env.TF_VERSION }}

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}
```

```yaml
      - name: Terraform Init
        working-directory: ./terraform
        run: terraform init

      - name: Terraform Apply (Dev)
        working-directory: ./terraform
        env:
          TF_VAR_db_admin_password: ${{ secrets.DEV_DB_ADMIN_PASSWORD }}
          TF_VAR_app_user_password: ${{ secrets.DEV_APP_PASSWORD }}
        run: |
          terraform workspace select dev
          terraform apply -var-file=dev.tfvars -auto-approve

      - name: Notify Slack
        uses: 8398a7/action-slack@v3
        with:
          status: ${{ job.status }}
          text: 'Terraform applied to Dev environment'
          webhook_url: ${{ secrets.SLACK_WEBHOOK }}
        if: always()

  # Job 3: Применение на Prod (с manual approval)
  terraform-apply-prod:
    name: Apply to Production
    runs-on: ubuntu-latest
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    needs: terraform-apply-dev
    environment: production  # Требует manual approval в GitHub

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ env.TF_VERSION }}

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.PROD_AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.PROD_AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Terraform Init
        working-directory: ./terraform
        run: terraform init

      - name: Terraform Plan (Production)
        working-directory: ./terraform
```

```yaml
        env:
          TF_VAR_db_admin_password: ${{ secrets.PROD_DB_ADMIN_PASSWORD }}
          TF_VAR_app_user_password: ${{ secrets.PROD_APP_PASSWORD }}
        run: |
          terraform workspace select production
          terraform plan -var-file=prod.tfvars -no-color

      - name: Terraform Apply (Production)
        working-directory: ./terraform
        env:
          TF_VAR_db_admin_password: ${{ secrets.PROD_DB_ADMIN_PASSWORD }}
          TF_VAR_app_user_password: ${{ secrets.PROD_APP_PASSWORD }}
        run: |
          terraform workspace select production
          terraform apply -var-file=prod.tfvars -auto-approve

      - name: Notify Slack (Success)
        uses: 8398a7/action-slack@v3
        with:
          status: 'success'
          text: '✅ Terraform successfully applied to Production!'
          webhook_url: ${{ secrets.SLACK_WEBHOOK }}
        if: success()

      - name: Notify Slack (Failure)
        uses: 8398a7/action-slack@v3
        with:
          status: 'failure'
          text: '❌ Terraform FAILED on Production! @channel'
          webhook_url: ${{ secrets.SLACK_WEBHOOK }}
        if: failure()
```

Настройка GitHub Secrets:

```
Settings → Secrets and variables → Actions → New repository secret

# AWS Credentials
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
PROD_AWS_ACCESS_KEY_ID
PROD_AWS_SECRET_ACCESS_KEY

# Database Passwords
DEV_DB_ADMIN_PASSWORD
DEV_APP_PASSWORD
PROD_DB_ADMIN_PASSWORD
PROD_APP_PASSWORD

# Slack Webhook
SLACK_WEBHOOK
```

Настройка Environments для manual approval:

```
Settings → Environments → New environment

Name: production
✅ Required reviewers: [выберите пользователей]
```

1.9.3 Интеграция с миграциями баз данных

Проблема: Terraform управляет инфраструктурой (роли, базы), но не схемами таблиц (это задача миграций).

Решение: Terraform + Flyway/Liquibase

Workflow:

1. Terraform создаёт базу данных и роли
2. Flyway/Liquibase применяет SQL-миграции (CREATE TABLE, ALTER, и т.д.)

Пример с Flyway:

```yaml
# .github/workflows/terraform-and-migrations.yml

jobs:
  terraform-apply:
    name: Apply Terraform
    runs-on: ubuntu-latest
    outputs:
      db_endpoint: ${{ steps.terraform.outputs.db_endpoint }}
    steps:
      # ... (Terraform steps как выше) ...

      - name: Get DB Endpoint
        id: terraform
        working-directory: ./terraform
        run: |
          DB_ENDPOINT=$(terraform output -raw db_instance_endpoint)
          echo "db_endpoint=$DB_ENDPOINT" >> $GITHUB_OUTPUT

  flyway-migrate:
    name: Run Flyway Migrations
    runs-on: ubuntu-latest
    needs: terraform-apply

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Flyway
        uses: joshuaavalon/flyway-action@v3.0.0
        with:
          url: jdbc:postgresql://${{ needs.terraform-
        apply.outputs.db_endpoint }}/myapp
          user: ${{ secrets.DB_ADMIN_USER }}
          password: ${{ secrets.DB_ADMIN_PASSWORD }}
        env:
          FLYWAY_LOCATIONS: filesystem:./migrations

      - name: Run Migrations
        run: flyway migrate
```

Структура проекта:

```
project/
├── terraform/
|   ├── main.tf
|   └── ...
├── migrations/
|   ├── V1__create_users_table.sql
|   ├── V2__add_email_to_users.sql
|   └── V3__create_orders_table.sql
└── .github/
    └── workflows/
        └── terraform-and-migrations.yml
```

1.10 Практические рецепты и шаблоны

1.10.1 Рецепт 1: Автоматическое создание расширений для новых баз

Сценарий: Каждая новая база должна иметь стандартный набор расширений.

```
# modules/database-with-extensions/main.tf

locals {
  # Стандартные расширения для всех баз
  standard_extensions = ["pg_stat_statements", "pgcrypto", "pg_trgm"]

  # Объединить стандартные + дополнительные из переменной
  all_extensions = distinct(concat(local.standard_extensions,
var.additional_extensions))
}

resource "postgresql_database" "this" {
  name = var.database_name
  # ...
}

resource "postgresql_extension" "standard" {
  for_each = toset(local.all_extensions)

  name     = each.value
  database = postgresql_database.this.name
  schema   = "public"
}
```

Использование:

```
module "analytics_db" {
  source = "./modules/database-with-extensions"

  database_name = "analytics"
  additional_extensions = ["timescaledb", "postgis"]
  # Итого: pg_stat_statements, pgcrypto, pg_trgm, timescaledb, postgis
}
```

1.10.2 Рецепт 2: Генерация случайных паролей с сохранением в AWS Secrets Manager

```
# main.tf

# Генерация пароля
resource "random_password" "app_user" {
  length  = 32
  special = true
}

# Роль с сгенерированным паролем
resource "postgresql_role" "app_user" {
  name     = "app_user"
  login    = true
  password = random_password.app_user.result
}

# Сохранение в AWS Secrets Manager
resource "aws_secretsmanager_secret" "app_user_password" {
  name = "myapp/database/app_user_password"
  description = "Password for PostgreSQL app_user role"

  recovery_window_in_days = 30  # Защита от случайного удаления
}

resource "aws_secretsmanager_secret_version" "app_user_password" {
  secret_id     = aws_secretsmanager_secret.app_user_password.id
  secret_string = jsonencode({
    username = postgresql_role.app_user.name
    password = random_password.app_user.result
    engine   = "postgres"
    host     = var.db_host
    port     = 5432
    dbname   = var.db_name
  })
}
```

Получение пароля из Secrets Manager:

```
# CLI
aws secretsmanager get-secret-value --secret-id myapp/database/
        app_user_password | jq -r '.SecretString'

# Python (boto3)
import boto3
import json

client = boto3.client('secretsmanager')
response = client.get_secret_value(SecretId='myapp/database/app_user_password')
secret = json.loads(response['SecretString'])

connection_string = f"postgresql://{secret['username']}:{secret['password']}
        @{secret['host']}:{secret['port']}/{secret['dbname']}"
```

1.10.3 Рецепт 3: Автоматическая ротация паролей

Terraform + AWS Lambda для ротации:

```
# rotation.tf

# Secret с ротацией
resource "aws_secretsmanager_secret" "db_master_password" {
  name = "myapp/database/master_password"

  # Ротация каждые 30 дней
  rotation_rules {
    automatically_after_days = 30
  }
}

# Lambda для ротации
resource "aws_secretsmanager_secret_rotation" "db_master" {
  secret_id           = aws_secretsmanager_secret.db_master_password.id
  rotation_lambda_arn = aws_lambda_function.rotate_db_password.arn

  rotation_rules {
    automatically_after_days = 30
  }
}

# Lambda function (упрощённо)
resource "aws_lambda_function" "rotate_db_password" {
  filename      = "lambda_rotation.zip"
  function_name = "rotate-postgres-password"
  role          = aws_iam_role.lambda_rotation.arn
  handler       = "index.handler"
  runtime       = "python3.11"

  environment {
    variables = {
      DB_ENDPOINT = aws_db_instance.postgresql.endpoint
    }
  }
}

# IAM Role для Lambda
resource "aws_iam_role" "lambda_rotation" {
  name = "lambda-db-password-rotation"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "lambda.amazonaws.com"
      }
    }]
  })
```

```
}

# Политика: доступ к Secrets Manager и RDS
resource "aws_iam_role_policy_attachment" "lambda_secrets" {
  role       = aws_iam_role.lambda_rotation.name
  policy_arn = "arn:aws:iam::aws:policy/SecretsManagerReadWrite"
}
```

Lambda код (Python):

```python
# lambda_rotation/index.py

import boto3
import psycopg2
import os
import json

secrets_client = boto3.client('secretsmanager')

def handler(event, context):
    # AWS Secrets Manager вызывает Lambda с событиями:
    # createSecret, setSecret, testSecret, finishSecret

    token = event['Token']
    step = event['Step']
    secret_arn = event['SecretId']

    if step == "createSecret":
        create_secret(secret_arn, token)
    elif step == "setSecret":
        set_secret(secret_arn, token)
    elif step == "testSecret":
        test_secret(secret_arn, token)
    elif step == "finishSecret":
        finish_secret(secret_arn, token)

def create_secret(secret_arn, token):
    # Генерируем новый пароль
    new_password = generate_password()

    # Сохраняем как pending secret
    secrets_client.put_secret_value(
        SecretId=secret_arn,
        ClientRequestToken=token,
        SecretString=json.dumps({"password": new_password}),
        VersionStages=['AWSPENDING']
    )

def set_secret(secret_arn, token):
    # Применяем новый пароль в PostgreSQL
    current_secret = get_secret_version(secret_arn, "AWSCURRENT")
    pending_secret = get_secret_version(secret_arn, "AWSPENDING")

    # Подключаемся с текущим паролем
    conn = psycopg2.connect(
        host=os.environ['DB_ENDPOINT'].split(':')[0],
        database='postgres',
        user='postgres',
        password=current_secret['password']
    )
```

```
    # Меняем пароль
    cur = conn.cursor()
    cur.execute(f"ALTER USER postgres WITH PASSWORD %s;",
        (pending_secret['password'],))
    conn.commit()
    conn.close()

def test_secret(secret_arn, token):
    # Проверяем, что новый пароль работает
    pending_secret = get_secret_version(secret_arn, "AWSPENDING")

    conn = psycopg2.connect(
        host=os.environ['DB_ENDPOINT'].split(':')[0],
        database='postgres',
        user='postgres',
        password=pending_secret['password']
    )
    conn.close()

def finish_secret(secret_arn, token):
    # Переключаем AWSPENDING → AWSCURRENT
    secrets_client.update_secret_version_stage(
        SecretId=secret_arn,
        VersionStage='AWSCURRENT',
        MoveToVersionId=token,
        RemoveFromVersionId=get_current_version_id(secret_arn)
    )

def generate_password():
    import secrets
    import string
    alphabet = string.ascii_letters + string.digits + "!@#$%^&*"
    return ''.join(secrets.choice(alphabet) for _ in range(32))

def get_secret_version(secret_arn, stage):
    response = secrets_client.get_secret_value(SecretId=secret_arn,
        VersionStage=stage)
    return json.loads(response['SecretString'])

def get_current_version_id(secret_arn):
    response = secrets_client.describe_secret(SecretId=secret_arn)
    return response['VersionIdsToStages']['AWSCURRENT'][0]
```

1.10.4 Рецепт 4: Импорт существующих ресурсов в Terraform

Сценарий: У вас уже есть база данных и роли, созданные вручную. Вы хотите управлять ими через Terraform.

Шаг 1: Написать конфигурацию

```
# main.tf

resource "postgresql_role" "existing_user" {
  name     = "app_user"
  login    = true
  password = "temporary"  # Будет перезаписано при import
}

resource "postgresql_database" "existing_db" {
  name = "myapp"
}
```

Шаг 2: Импортировать ресурсы

```
# Импорт роли
terraform import postgresql_role.existing_user app_user

# Импорт базы данных
terraform import postgresql_database.existing_db myapp

# Terraform создаст State с этими ресурсами
```

Шаг 3: Синхронизировать конфигурацию с реальным состоянием

```
# Terraform покажет drift (различия)
terraform plan

# Пример вывода:
# ~ resource "postgresql_role" "existing_user" {
#     ~ connection_limit = -1 -> 20  # В коде 20, а в реальности -1
#   }

# Исправьте конфигурацию:
# connection_limit = -1  # Как в реальной БД
```

Шаг 4: Применить (должно быть “No changes”)

```
terraform plan
# No changes. Infrastructure is up-to-date.
```

1.10.5 Рецепт 5: Управление множеством баз через CSV

Сценарий: У вас 100 микросервисов, каждый нужна база. Хранить список в CSV.

Файл: `databases.csv`

```
service_name,connection_limit,extensions
user-service,50,"pgcrypto,pg_trgm"
order-service,30,pgcrypto
inventory-service,20,"pgcrypto,pg_stat_statements"
analytics-service,100,"timescaledb,postgis"
```

Terraform:

```
# main.tf

locals {
  # Читаем CSV
  databases_csv = csvdecode(file("${path.module}/databases.csv"))

  # Преобразуем в map для for_each
  databases = {
    for db in local.databases_csv :
    db.service_name => {
      connection_limit = tonumber(db.connection_limit)
      extensions       = split(",", db.extensions)
    }
  }
}

# Создаём базы
resource "postgresql_database" "services" {
  for_each = local.databases

  name             = replace(each.key, "-", "_")  # user-service → user_service
  connection_limit = each.value.connection_limit
}

# Создаём роли
resource "postgresql_role" "services" {
  for_each = local.databases

  name     = "${replace(each.key, "-", "_")}_role"
  login    = true
  password = random_password.services[each.key].result
}

# Генерация паролей
resource "random_password" "services" {
  for_each = local.databases

  length  = 32
  special = true
}

# Расширения
resource "postgresql_extension" "services" {
  for_each = {
    for item in flatten([
      for service, config in local.databases : [
        for ext in config.extensions : {
          key      = "${service}-${ext}"
          service  = service
          ext_name = ext
        }
```

```
      ]
    ]) : item.key => item
  }

  name     = each.value.ext_name
  database = postgresql_database.services[each.value.service].name
}
```

Результат:

```
terraform apply

# Создаётся:
# - 4 базы данных
# - 4 роли с паролями
# - 8 расширений
```

Добавить новый сервис:

```
# databases.csv
...,
notification-service,15,pgcrypto
```

```
terraform plan
# + 1 database, + 1 role, + 1 extension to add

terraform apply
```

1.11 Troubleshooting и подводные камни

1.11.1 Проблема 1: “Error: pq: password authentication failed”

Причина: Неверный пароль для подключения к PostgreSQL.

Решение:

1. Проверьте переменную:

```
echo $TF_VAR_db_admin_password
```

1. Убедитесь, что пароль правильный:

```
psql -h localhost -U postgres -c "SELECT 1"
# Введите пароль
```

1. Проверьте `pg_hba.conf` (метод аутентификации):

```
# Должно быть:
host    all    all    0.0.0.0/0    md5  # или scram-sha-256
```

1.11.2 Проблема 2: “Error: role already exists”

Причина: Роль создана вручную, а Terraform пытается её создать.

Решение: Импортировать существующую роль

```
terraform import postgresql_role.app_user app_user
```

1.11.3 Проблема 3: Drift Detection — ресурс изменён вручную

Сценарий:

```
# Кто-то вручную изменил connection_limit
psql -c "ALTER ROLE app_user CONNECTION LIMIT 100;"

# Terraform обнаруживает drift
terraform plan
# ~ resource "postgresql_role" "app_user" {
#     ~ connection_limit = 20 -> 100
#   }
```

Решение 1: Применить конфигурацию Terraform (вернуть к 20)

```
terraform apply
```

Решение 2: Обновить конфигурацию под реальность

```
resource "postgresql_role" "app_user" {
  connection_limit = 100  # Обновили код
}
```

Решение 3: Игнорировать изменения (lifecycle ignore_changes)

```
resource "postgresql_role" "app_user" {
  name             = "app_user"
  connection_limit = 20

  lifecycle {
    ignore_changes = [connection_limit]  # Terraform не будет управлять этим
  }
}
```

1.11.4 Проблема 4: State Lock — кто-то забыл разблокировать

Причина: Terraform процесс был убит (Ctrl+C, crash), lock не снялся.

Ошибка:

```
Error: Error acquiring the state lock
Lock ID: abc123-456def
Who: andrey@hostname
```

Решение: Принудительно разблокировать (ОПАСНО!)

```
# Убедитесь, что никто не работает с Terraform!
terraform force-unlock abc123-456def

# Terraform попросит подтверждение
# Do you really want to force-unlock?
#   Enter a value: yes
```

⚠️ **Внимание:** Используйте `force-unlock` только если уверены, что другой процесс НЕ работает!

1.11.5 Проблема 5: Циклические зависимости

Пример:

```
resource "postgresql_role" "app" {
  name = "app"
  # Роль принадлежит базе
}

resource "postgresql_database" "mydb" {
  name  = "mydb"
  owner = postgresql_role.app.name  # База зависит от роли
}

resource "postgresql_grant" "app" {
  database = postgresql_database.mydb.name  # Grant зависит от базы
  role     = postgresql_role.app.name
  privileges = ["CONNECT"]
}

# Если ещё добавить:
resource "postgresql_role" "app" {
  # ...
  depends_on = [postgresql_database.mydb]  # ❌ Циклическая зависимость!
}
```

Ошибка:

```
Error: Cycle: postgresql_role.app, postgresql_database.mydb
```

Решение: Убрать избыточные зависимости

Terraform автоматически определяет зависимости через ссылки (`postgresql_role.app.name`). Не добавляйте `depends_on` без необходимости.

1.11.6 Проблема 6: Terraform хочет пересоздать ресурс (- / +)

Причина: Некоторые атрибуты нельзя изменить in-place, требуется пересоздание.

Пример:

```
resource "postgresql_database" "mydb" {
  name      = "mydb"
  encoding  = "UTF8"
}

# Изменили encoding
resource "postgresql_database" "mydb" {
  name      = "mydb"
  encoding  = "LATIN1"  # Нельзя изменить без пересоздания!
}
```

Terraform Plan:

```
-/+ resource "postgresql_database" "mydb" {
      ~ encoding = "UTF8" -> "LATIN1"
      ...
    }

# Plan: 1 to add, 0 to change, 1 to destroy.
```

⚠️ **ОПАСНО!** Пересоздание базы = потеря данных!

Решение:

1. Откатить изменение в коде (оставить encoding = “UTF8”)
2. Если действительно нужно изменить encoding — делать через dump/restore, а не Terraform

1.11.7 Проблема 7: “Error: extension does not exist”

Причина: Расширение не установлено на сервере PostgreSQL.

Ошибка:

```
Error: pq: extension "timescaledb" is not available
```

Решение:

1. Установите расширение на сервер:

```
# Для RDS: добавить в Parameter Group
shared_preload_libraries = 'timescaledb'

# Для самоуправляемого PostgreSQL:
apt-get install postgresql-16-timescaledb
```

1. Перезапустите PostgreSQL

2. Применить Terraform снова

1.11.8 Проблема 8: Secrets в логах Terraform

Проблема: Пароли видны в `terraform plan` output.

Решение: Используйте `sensitive = true` для переменных

```
variable "db_password" {
  type      = string
  sensitive = true  # Terraform замаскирует значение
}
```

Output:

```
# Вместо:
password = "MySecretPassword123"

# Будет:
password = (sensitive value)
```

⚠️ **НО:** State файл всё равно содержит пароль в открытом виде! Обязательно используйте remote backend с шифрованием.

1.11.9 Проблема 9: Изменение пароля не применяется

Сценарий:

```
resource "postgresql_role" "app_user" {
  name     = "app_user"
  password = "old_password"
}

# Изменили на new_password
resource "postgresql_role" "app_user" {
  name     = "app_user"
  password = "new_password"
}
```

Terraform Plan:

```
No changes. Infrastructure is up-to-date.
```

Причина: PostgreSQL провайдер по умолчанию не обновляет пароль если роль уже существует (во избежание случайных изменений).

Решение: Использовать `password` с `lifecycle`

```
resource "postgresql_role" "app_user" {
  name     = "app_user"
  password = var.app_user_password

  # Принудительно обновлять пароль при изменении
  lifecycle {
    ignore_changes = []  # Не игнорировать password
  }
}
```

Или использовать `random_password` с `keepers`:

```
resource "random_password" "app_user" {
  length  = 32

  keepers = {
    # Изменение этого значения приведёт к генерации нового пароля
    rotation = "2026-02-13"
  }
}

resource "postgresql_role" "app_user" {
  name     = "app_user"
  password = random_password.app_user.result
}
```

1.12 Полезные ресурсы

1.12.1 Официальная документация

Terraform:

- Terraform Documentation
- HashiCorp Learn Terraform
- Terraform AWS Provider
- Terraform PostgreSQL Provider

PostgreSQL:

- PostgreSQL Official Documentation
- PostgreSQL pg_hba.conf

- PostgreSQL Roles and Privileges

1.12.2 Книги

Infrastructure as Code:

- “Terraform: Up & Running” by Yevgeniy Brikman (O’Reilly, 2022)
- “Infrastructure as Code” by Kief Morris (O’Reilly, 2021)

PostgreSQL:

- “PostgreSQL 14 Administration Cookbook” by Simon Riggs, Gianni Ciolli (Packt, 2021)
- “Mastering PostgreSQL 15” by Hans-Jürgen Schönig (Packt, 2023)

1.12.3 Блоги и статьи

- Gruntwork Blog — Terraform Best Practices
- HashiCorp Blog
- PostgreSQL Weekly Newsletter
- Percona Database Blog

1.12.4 Инструменты и утилиты

Terraform Tooling:

- tflint — Линтер для Terraform
- terraform-docs — Автогенерация документации
- checkov — Security scanning для IaC
- Infracost — Оценка стоимости инфраструктуры

Database Migrations:

- Flyway — Database migration tool
- Liquibase — Database schema change management
- Alembic — Database migration for Python (SQLAlchemy)

PostgreSQL Tools:

- pgAdmin — GUI для PostgreSQL
- DBeaver — Universal database tool
- pg_stat_statements — Query performance monitoring

1.12.5 Сообщества

- Terraform Community Forum

- PostgreSQL Mailing Lists
- Reddit r/Terraform
- Reddit r/PostgreSQL
- PostgreSQL Slack

1.12.6 Курсы

- HashiCorp Certified: Terraform Associate
- Terraform on AWS (Udemy)
- PostgreSQL Advanced (OTUS)

1.13 Заключение

1.13.1 Что вы теперь знаете

После изучения этого дополнительного материала вы освоили:

✅ **Парадигму Infrastructure as Code** — декларативное управление инфраструктурой через код

✅ **Terraform основы** — синтаксис HCL, ресурсы, переменные, outputs, modules

✅ **Управление PostgreSQL в облаках** — автоматическое создание RDS, Azure Database, GCP CloudSQL через Terraform

✅ **PostgreSQL Provider** — управление ролями, базами, схемами, расширениями, правами доступа

✅ **Модули и переиспользование кода** — создание универсальных шаблонов для стандартных задач

✅ **State Management** — безопасное хранение State в S3, работа в команде, locking, versioning

✅ **CI/CD для баз данных** — автоматизация применения изменений через GitOps, code review, approval gates

✅ **Практические рецепты** — автоматическая генерация паролей, импорт существующих ресурсов, управление через CSV

✅ **Troubleshooting** — решение типичных проблем (State locks, drift detection, циклические зависимости)

1.13.2 Преимущества подхода IaC для PostgreSQL DBA

Было (без Terraform):

- Ручное создание баз и ролей через SQL-скрипты
- Различия между prod, staging, dev
- Нет версионирования изменений
- Трудно масштабировать (50 баз = 50 раз выполнить вручную)
- Человеческий фактор: опечатки, забытые команды

Стало (с Terraform):

- ✅ Одна декларативная конфигурация — все окружения идентичны
- ✅ Версионирование в Git — полная история изменений
- ✅ Code review — каждое изменение проверяется командой
- ✅ Автоматизация — CI/CD применяет изменения автоматически
- ✅ Масштабирование — один модуль = 100 баз
- ✅ Аудит — кто, когда, что изменил (Git log)
- ✅ Откат — `git revert` + `terraform apply`
- ✅ Безопасность — пароли в Secrets Manager, State зашифрован

1.13.3 Что дальше

Следующие шаги в вашем пути:

1. **Практика:** Начните с простого проекта — создайте базу данных и роль через Terraform на локальном PostgreSQL
2. **Миграция существующей инфраструктуры:** Импортируйте существующие ресурсы в Terraform (`terraform import`)
3. **Модули:** Создайте свой первый модуль для типовых задач (база + роль + права)
4. **Remote Backend:** Настройте S3 + DynamoDB для безопасного хранения State
5. **CI/CD:** Интегрируйте Terraform в ваш CI/CD pipeline (GitHub Actions, GitLab CI, Jenkins)
6. **Advanced Topics:**
    - Terraform Workspaces для управления множеством окружений
    - Dynamic Blocks для гибкой конфигурации
    - Custom Providers для специфичных задач
    - Terraform Cloud для коллаборации в команде

7. **Комбинация с миграциями:** Объедините Terraform (инфраструктура) + Flyway/Liquibase (схемы)
8. **Мониторинг и алерты:** Добавьте CloudWatch / Prometheus для мониторинга созданных ресурсов

На следующем занятии PostgreSQL Advanced курса:

Мы углубимся в архитектуру PostgreSQL, оптимизацию производительности, репликацию и высокую доступность. Terraform станет инструментом для автоматизации развёртывания HA-кластеров PostgreSQL.

Полезные команды для быстрого старта:

```
# Установка Terraform
brew install terraform  # MacOS
# или скачайте с terraform.io

# Инициализация проекта
terraform init

# Проверка конфигурации
terraform validate

# Предпросмотр изменений
terraform plan

# Применение изменений
terraform apply

# Откат (destroy)
terraform destroy

# Импорт существующих ресурсов
terraform import postgresql_role.app_user app_user

# Форматирование кода
terraform fmt -recursive

# Просмотр State
terraform show

# Список workspaces
terraform workspace list

# Принудительная разблокировка
terraform force-unlock <lock-id>
```

Этот дополнительный материал подготовлен для курса “PostgreSQL Advanced” OTUS.
Автор: Дмитрий Золотов Дата: 2026-02-13

Удачи в освоении Infrastructure as Code и Terraform для PostgreSQL! 🚀